Solution Summary: Classify Math Publications

Redwanul Karim

March 22, 2025

Nomenclature

$d^{(i)} \in \mathcal{D}$	The i-th document in the corpus
$f_j^{(i)} \in \mathcal{F}$	The j-th formula within document i
$\mathcal{V}$	Token vocabulary of size 7500
Δ^{n-1}	$(n-1)$-dimensional probability simplex
$\mathbb{S}$	Unicode math symbol set (`e.g.`, $\sigma, \sum, \sqrt[3]{x}, \forall, \exists, \in, \nabla, \doteq, \ldots$ `etc.`)
τ	Frequency threshold (in our case, $\tau = 2$)
$[t]_k$	The k-th token in a tokenized sequence
[CLS]	Classification token embedding
[BOS]	Beginning of a formula
[EOS]	End of a formula

Problem Formalization

We are given a collection of documents $\mathcal{D} = \left\{d^{(1)}, d^{(2)}, \ldots, d^{(n)}\right\}$, each associated with a category label

$$y^{(i)} \in \{1, \ldots, 18\}.$$

The objective is to learn a function

$$f : \mathcal{D} \to \Delta^{17} \subset \mathbb{R}^{18}, \quad \texttt{where}, \quad \Delta^{17} = \left\{ (p_1, p_2, \ldots, p_{18}) \in \mathbb{R}^{18} \;\middle|\; \sum_{c=1}^{18} p_c = 1, \quad p_c \geq 0 \text{ for all } c \right\}$$

Hence, for any new document d, the model f should produce an 18-dimensional vector of probabilities over the 18 possible classes.

Data Preparation

Accurate math document classification requires carefully designed tokenization to handle the rich variety of symbols, structures, and tags found in mathematical content (e.g., formula markup in `MathML`). We therefore propose a multi-stage tokenization pipeline focused on formula segments.

Formula Tokenization Pipeline

Let F_d be the set of all formulas extracted from a document d. For each formula $f \in F_d$, the following steps are performed:

1. **XML Tag Isolation:**

   We first split the formula into a list of tokens based on standard MathML/XML tags $\mathcal{XML}$ (e.g., `<mrow>`, `<mi>`, `<mo>`). Formally:

   $$\texttt{Split}(f) \;=\; \bigoplus_{t \in \texttt{split}(f)} \begin{cases} t, & \text{if } t \in \mathcal{XML}, \\ \texttt{ByteLevel}(t), & \text{otherwise.} \end{cases}$$

   That is, if t is a recognized XML tag, we keep it as a standalone token without its attributes added from the MathML parsing; otherwise, we further break t into byte-level tokens to handle arbitrary strings and symbols.

2. **Number Tokenization:**

   Given a numeric string $n \in \mathbb{N}^* \cup \mathbb{R}$, we define a function Tokenize(n) that decomposes n into a structured sequence of tokens based on its format. Formally,

$$\text{Tokenize}(n) \;=\; \bigoplus_{t\in\text{split}(n)} \begin{cases} \texttt{[NEG]}, & \text{if } n \text{ starts with "}-\text{"}, \\ \texttt{[SCI]} \;\oplus \text{Tokenize(base)} \oplus \texttt{[EXP]} \;\oplus \text{Tokenize(exp)}, & \text{if } n = \text{base} \times 10^{\text{exp}}, \\ \texttt{[FLOAT]} \oplus \text{Tokenize(int)} \;\oplus \texttt{[DECIMAL]} \oplus \text{Tokenize(frac)}, & \text{if } n = \texttt{int}.\text{frac}, \\ \texttt{[INT]} \;\oplus n, & \text{otherwise.} \end{cases}$$

`where,` $\oplus$ `denotes token concatenation.`

This ensures that each numeric value is not considered as separate tokens but is consistently tokenized based on its structure, enabling efficient parsing and model processing.

3. **Byte-Pair Encoding (BPE) Construction:**
   Let $\mathcal{V}_0$ be our initial vocabulary containing meta-tokens (`[CLS]`, `[PAD]`, `[UNK]`, `[BOS]`, `[EOS]`, `[INT]`, `[FLOAT]`, ... `etc.`) and all sufficiently frequent symbols $s \in \mathbb{S}$ with $\text{count}(s) \geq \tau$ (where $\tau = 2$). Recursively, we expand the vocabulary by identifying the most frequently co-occurring token pairs:
   $$\mathcal{V}_{k+1} = \mathcal{V}_k \;\cup\; \Big\{ x \oplus y \;\Big|\; (x,y) = \arg\max_{(a,b)} \sum_{f\in\mathcal{F}} N_{ab}(f) \Big\},$$
   where $N_{ab}(f)$ counts the occurrences of the bigram (a,b) in f. We repeat this merge operation until $|\mathcal{V}| = 7500$, ensuring that common symbol pairs become single tokens in the final vocabulary.

4. **Token Encoding:**
   Once the vocabulary is constructed, each formula f is transformed into a sequence of IDs as follows:
   $$\text{Enc}(f) \;=\; \texttt{[BOS]} \;\oplus\; \bigoplus_{i=1}^{l} \begin{cases} \mathcal{V}^{-1}(t_i), & t_i \in \mathcal{V}, \\ \texttt{[UNK]}, & \text{otherwise.} \end{cases} \;\oplus \texttt{[EOS]}$$
   In addition to `[BOS]` and `[EOS]` token, the classification token `[CLS]` is also added at the beginning of every formula sequence inside the model definition to allow the model to learn a representative embedding for that formula.

This pipeline handles diverse MathML structures, preserves important symbolic tokens, and adapts naturally to rare or previously unseen symbols via the BPE merges.

Model Architecture

We adopt a *hierarchical* multi-transformer approach that first encodes formula-level contexts and then aggregates these intermediate embeddings to generate a comprehensive document-level representation.

Formula-Level Transformer

Each formula f_j in a document d is tokenized into $[t_1, t_2, \ldots, t_L]$. We introduce a learnable embedding for the classification token, $\texttt{[CLS]}^{(e)}$. Let $W_e \in \mathbb{R}^{|\mathcal{V}|\times 256}$ be the formula token embedding matrix and $p_i \in \mathbb{R}^{256}$ the positional embedding at index i. The initial embeddings for layer $l = 0$ are

$$E_j^{(0)} \;=\; \texttt{[CLS]}^{(e)} \;\oplus\; \big(W_e\, t_1 + p_1,\; W_e\, t_2 + p_2,\; \ldots,\; W_e\, t_L + p_L\big).$$

A 3-layer transformer block with multi-head self-attention is then applied:

$$E_j^{(l)} = \texttt{TransformerBlock}\big(E_j^{(l-1)}\big), \quad \text{for } l = 1, \ldots, 3.$$

The final formula-level hidden state is the embedding at the classification token position of the 4th layer output:

$$h_j = E_j^{(3)}[\texttt{[CLS]}] \;\in\; \mathbb{R}^{256}.$$

Hence, h_j serves as a condensed vector representation of the entire formula f_j.

Document-Level Transformer

After obtaining formula embeddings $\{h_1, \ldots, h_m\}$ for all m formulas in a document (we fix $m = 10$), we stack them into a sequence and prepend a distinct learnable document classification token, $\texttt{[CLS]}^{(d)}$:

$$H^{(0)} = \texttt{[CLS]}^{(d)} \;\oplus\; [\,h_1,\, h_2, \ldots, h_{10}\,].$$

We then apply a 4-layer transformer, each layer containing a multi-head attention mechanism (with 8 attention heads) and a position-wise feed-forward network:

$$H^{(l)} = \texttt{TransformerBlock}\big(H^{(l-1)}\big), \quad \text{for } l = 1, \ldots, 4.$$

The output $H^{(4)}[\texttt{[CLS]}] \in \mathbb{R}^{256}$ is the final aggregated representation of the entire document. We denote this vector by

$$z = H^{(4)}[\texttt{[CLS]}].$$

Classification Head

The classification head predicts the probability distribution over the 18 classes for the document. It consists of a two-layer feed-forward network with a `GELU` activation, layer normalization on z, and a final softmax:

$$P(y \mid d) \;=\; \texttt{Softmax}\big(W_2\,\sigma\big(W_1\big[z \oplus \texttt{LayerNorm}(z)\big] + b_1\big) + b_2\big).$$

Here, $W_1 \in \mathbb{R}^{256\times 256}$, $W_2 \in \mathbb{R}^{256\times 18}$, and σ is the `GELU` nonlinearity. The softmax ensures the output is in Δ^{17} (the 17-simplex in 18-dimensional space).

Training Protocol

Let B be the batch size and θ be the set of all trainable parameters of the model (including embeddings, transformer weights, etc.). The training objective is:

$$\mathcal{L} \;=\; \frac{1}{B}\sum_{i=1}^{B} \texttt{CrossEntropy}\big(f(d^{(i)}), y^{(i)}\big) \;+\; \lambda\|\theta\|^2,$$

where λ is an L2 regularization coefficient (often folded into the weight-decay hyperparameter).
We use AdamW with decoupled weight decay:

$$\theta_{t+1} \;\leftarrow\; \theta_t \;-\; \eta_t\,\big(\nabla_\theta \mathcal{L} + 2\,\gamma\,\theta_t\big),$$

where γ is the weight decay rate and η_t is the learning rate at step t. The learning rate is scheduled via a warmup-cosine schedule, simplified as:

$$\eta_t = 3\times 10^{-4} \;\cdot\; \min\big(t^{-0.5},\; t\cdot T^{-1.5}\big) \quad \text{with} \quad T = 20.$$

In practice, we set $\lambda = 0.1$, $\gamma = 0.01$, and apply standard settings $\beta_1 = 0.9$, $\beta_2 = 0.98$ for AdamW. A dropout of 0.4 is used in all layers to prevent overfitting.

Experimental Results

We trained the model for 150 epochs using a cosine learning rate schedule, which gradually adjusts the learning rate over time to improve convergence and prevent overshooting. After training, the model achieved a mean classification accuracy of **64.45%**, demonstrating its ability to effectively learn patterns from the dataset.

Training Configuration

- **Optimizer:** AdamW with $(\beta_1 = 0.9, \beta_2 = 0.98)$
- **Learning Rate:** 3×10^{-4} peak, with warmup and cosine decay
- **Batch Size:** 256 documents per step
- **Dropout:** 0.4 on all layers
- **Weight Decay:** 0.01
- **Implementation:** PyTorch with MLflow for experiment tracking

Conclusion

We propose a hierarchical transformer architecture for mathematical document classification that integrates a specialized tokenization pipeline for mathematical formulas. The main contributions and findings are:

- **Math-Aware Tokenization:** A carefully designed BPE approach captures fine-grained math symbols and common patterns, while still allowing for open-ended vocab expansion.
- **Hierarchical Transformer Encoding:** By first encoding each formula with a 3-layer transformer and then applying a 4-layer transformer over the resulting embeddings, we effectively learn cross-formula and overall document relationships.
- **Robust Experimental Performance:** Achieving **64.45%** accuracy on an arXiv-based test set, with particularly strong differentiation among major math and astro-physics categories.

In future work, we plan to extend these ideas by incorporating graph-based representations of formula parse trees and allowing dynamic document lengths rather than fixed sampling of formulas.